

CHAPTER- IV

OOPS INTRODUCTION AND BASICS

OOPS (Object- Oriented Programming System):

- It is a programming approach where we design programs using objects that represent real-world entities.
- Each object represents a real world entity.
- It has attributes(data and variables) and behaviours(methods and functions).

NECESSITY OF OOPS:

- Real world modelling
- Code reusability
- Easy to maintain and update
- Data security using encapsulation
- Simplifies complex programs

OOPS CONCEPTS:

- Class
- Object
- Inheritance
- Encapsulation
- Polymorphism
- Abstraction

CLASS:

A class is blue print of an object.

EXAMPLES:

```
class Car { //class creation
    String brand;
    String model;
    double price;
    void display() {
        System.out.println("Brand:"+brand);
        System.out.println("Model:"+model);
        System.out.println("Price:"+price);
    }
}

public class CarMain{ //main class
    public static void main(String[] args) {
        Car c1= new Car();
        c1.brand= "Maruti Suzuki";
        c1.model="Swift";
        c1.price=579000;
        c1.display();
    }
}
```

OBJECT:

An object is an instances of a class.

SYNTAX:

ClassName ObjectName = new ClassName();

EXAMPLES:

```
class Tea{
    String t_name;
    String t_sub;
    double t_sal;
    void show() {
        System.out.println("Name:"+t_name);
        System.out.println("subject:"+t_sub);
        System.out.println("Salary:"+t_sal);
    }
}

public class TeaMain{
    public static void main(String[] args) {
        Tea t1= new Tea(); //object creation
        Scanner sc= new Scanner(System.in);
        System.out.println("enter the name");
        t1.t_name=sc.nextLine();
        System.out.println("enter the Subject");
        t1.t_sub=sc.nextLine();
    }
}
```

```
        System.out.println("enter the Salary");  
        t1.t_sal=sc.nextDouble();  
        t1.show();  
    }  
}
```

ENCAPSULATION:

- Encapsulation is the concept of wrapping data and methods.
- Encapsulation is used to hides data for security purpose.

ABSTRACTION:

- Abstraction is the concept of showing only essential details and hiding the inner details.
- Abstraction is achieved using **abstract** keyword.

INHERITANCES:

- Inheritances allows child class to inherits all its behaviour from their parents class.
- Example: Dog() class inherits its behaviour from Animal() class.

POLYMORPHISM:

- Polymorphism means “**many forms**”.
- **TYPES:**
 - ✓ Runtime polymorphism(method overriding).
 - ✓ Compile time polymorphism (method overloading).

